MATERIA
INF560 — Desarrollo Web Backend

GUÍA DE LABORATORIO 5

Modelos, Migraciones y Relaciones

Material de Estudio Teórico-Práctico

CONTENIDO DE LA UNIDAD

1. Modelos Eloquent — $fillable, $casts, accessors, boot
2. Relaciones Eloquent — belongsToMany, hasMany, belongsTo, hasOne
3. Factories y Seeders — datos realistas para pruebas
4. Tinker — exploración interactiva de relaciones
5. Ejercicios propuestos y Git tag v2.0

COMPETENCIA DE LA UNIDAD

El estudiante será capaz de diseñar e implementar modelos Eloquent con propiedades, accessors, mutators y relaciones (uno a uno, uno a muchos, muchos a muchos), poblar la base de datos con factories y seeders, y verificar la integridad del modelo relacional mediante la consola interactiva Tinker.

Objetivos de la Guía

Al finalizar esta guía, el estudiante será capaz de:

- Crear modelos Eloquent con propiedades $fillable, $casts y accessors personalizados.
- Implementar relaciones entre modelos: hasOne, hasMany, belongsTo y belongsToMany.
- Diseñar factories con datos realistas usando Faker.
- Crear seeders para poblar la base de datos con datos de prueba coherentes.
- Utilizar Tinker para explorar y verificar las relaciones del modelo.

Prerrequisitos

Antes de iniciar esta guía, asegúrate de tener:

> **Estado del proyecto**
>
> Proyecto library-app creado con Laravel 13 y PostgreSQL configurado.
>
> Todas las migraciones ejecutadas: users, authors, categories, books, author_book, members, loans (Guía 4).
>
> Git inicializado con tag v1.0.
>
> Servidor de desarrollo funcional con php artisan serve.

Desarrollo: Modelos Eloquent

En esta sección crearemos los modelos Eloquent que representan cada tabla de nuestra base de datos. Cada modelo define las propiedades de asignación masiva ($fillable), las conversiones de tipo ($casts), los accessors para atributos calculados y los métodos de lógica de negocio.

> **Modelo Eloquent**
> Un modelo Eloquent es una clase PHP que representa una tabla de la base de datos. Laravel utiliza el patrón Active Record: cada instancia del modelo corresponde a una fila de la tabla. El modelo permite consultar, insertar, actualizar y eliminar registros mediante una API expresiva y orientada a objetos.

Paso 1: Modelo Author

El modelo Author representa a los autores de los libros. Genera el modelo con Artisan:

```
php artisan make:model Author
```

Edita el archivo app/Models/Author.php con el siguiente contenido:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Author extends Model
{
    use HasFactory;

    protected $fillable = [
        'first_name',
        'last_name',
        'nationality',
        'birth_date',
        'biography',
    ];

    protected $casts = [
        'birth_date' => 'date',
    ];

    protected $appends = ['full_name'];

    /**
     * Accessor: nombre completo del autor.
     */
    public function getFullNameAttribute(): string
    {
        return "{$this->first_name} {$this->last_name}";
    }
}
```

$fillable

El arreglo $fillable define qué campos pueden asignarse de forma masiva (mass assignment). Esto protege contra la inserción de datos no autorizados cuando se usa Author::create([...]) o $author->fill([...]).

$casts

El arreglo $casts indica a Eloquent cómo convertir automáticamente los valores al leerlos o escribirlos. Por ejemplo, 'birth_date' => 'date' convierte el campo a una instancia de Carbon, permitiendo operaciones como $author->birth_date->age.

Accessor

Un accessor es un método que transforma un atributo al accederlo. Se define con la convención get{Atributo}Attribute(). El arreglo $appends permite que el accessor se incluya automáticamente en la serialización JSON del modelo.

Paso 2: Modelo Category

El modelo Category representa las categorías de los libros. Genera el modelo:

```
php artisan make:model Category
```

Edita app/Models/Category.php:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Support\Str;

class Category extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'slug',
        'description',
        'color',
    ];

    /**
     * Boot: genera slug automáticamente al crear.
     */
    protected static function boot(): void
    {
        parent::boot();

        static::creating(function (Category $category) {
            if (empty($category->slug)) {
                $category->slug = Str::slug($category->name);
            }
        });
    }
}
```

Método boot()

El método boot() permite registrar eventos del ciclo de vida del modelo. En este caso, usamos el evento 'creating' para generar automáticamente el slug a partir del nombre, garantizando que cada categoría tenga una URL amigable.

Paso 3: Modelo Book

El modelo Book es el más complejo de nuestra aplicación. Incluye SoftDeletes para borrado lógico, accessors para disponibilidad y métodos para gestionar copias:

```
php artisan make:model Book
```

Edita app/Models/Book.php:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Book extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = [
        'title', 'isbn', 'publisher', 'publish_year',
        'pages', 'language', 'description', 'cover_url',
        'total_copies', 'available_copies', 'status',
        'category_id',
    ];

    protected $casts = [
        'publish_year'     => 'integer',
        'pages'            => 'integer',
        'total_copies'     => 'integer',
        'available_copies' => 'integer',
    ];

    /**
     * Accessor: ¿tiene copias disponibles?
     */
    public function getIsAvailableAttribute(): bool
    {
        return $this->available_copies > 0;
    }

    /**
     * Decrementa copias disponibles al prestar.
     */
    public function decrementCopies(): bool
    {
        if ($this->available_copies <= 0) {
            return false;
        }
```

```
        $this->decrement('available_copies');
        $this->refresh();

        if ($this->available_copies === 0) {
            $this->update(['status' => 'unavailable']);
        }

        return true;
    }

    /**
     * Incrementa copias al devolver un libro.
     */
    public function incrementCopies(): void
    {
        if ($this->available_copies < $this->total_copies) {
            $this->increment('available_copies');
            $this->refresh();

            if ($this->status !== 'available') {
                $this->update(['status' => 'available']);
            }
        }
    }
}
```

> **SoftDeletes**
> El trait SoftDeletes permite borrado lógico: en lugar de eliminar el registro de la base de datos, se marca con una fecha en la columna deleted_at. Esto permite recuperar libros eliminados accidentalmente. Los modelos con SoftDeletes se excluyen automáticamente de las consultas normales.

Paso 4: Modelo Member

El modelo Member representa a los miembros de la biblioteca. Cada miembro está asociado a un usuario del sistema:

```
php artisan make:model Member
```

Edita app/Models/Member.php:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Member extends Model
{
    use HasFactory;

    protected $fillable = [
        'user_id', 'member_code', 'phone', 'address',
        'membership_type', 'membership_expires_at',
        'max_loans', 'is_active',
    ];
```

```
    protected $casts = [
        'membership_expires_at' => 'date',
        'is_active'             => 'boolean',
        'max_loans'             => 'integer',
    ];

    /**
     * Boot: genera member_code al crear.
     */
    protected static function boot(): void
    {
        parent::boot();

        static::creating(function (Member $member) {
            if (empty($member->member_code)) {
                $member->member_code = 'LIB-'
                    . date('Ymd') . '-'
                    . str_pad(
                        random_int(0, 9999), 4, '0', STR_PAD_LEFT
                    );
            }
        });
    }

    /**
     * Accessor: ¿la membresía está activa?
     */
    public function getIsMembershipActiveAttribute(): bool
    {
        return $this->is_active
            && ($this->membership_expires_at === null
                || $this->membership_expires_at->gte(now()));
    }
}
```

Paso 5: Modelo Loan

El modelo Loan gestiona los préstamos de libros. Incluye accessors para verificar si el préstamo está vencido y calcular los días restantes:

```
php artisan make:model Loan
```

Edita app/Models/Loan.php:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Loan extends Model
{
    use HasFactory;

    protected $fillable = [
        'book_id', 'member_id', 'loaned_by',
        'loan_date', 'due_date', 'returned_date',
        'status', 'notes',
    ];
```

```
    protected $casts = [
        'loan_date'     => 'date',
        'due_date'      => 'date',
        'returned_date' => 'date',
    ];

    /**
     * Accessor: ¿el préstamo está vencido?
     */
    public function getIsOverdueAttribute(): bool
    {
        return $this->returned_date === null
            && $this->due_date->lt(now());
    }

    /**
     * Accessor: días restantes (negativo si vencido).
     */
    public function getDaysRemainingAttribute(): int
    {
        if ($this->returned_date !== null) {
            return 0;
        }

        return (int) $this->due_date->diffInDays(
            now(), false
        );
    }
}
```

Paso 6: Modelo User (actualización)

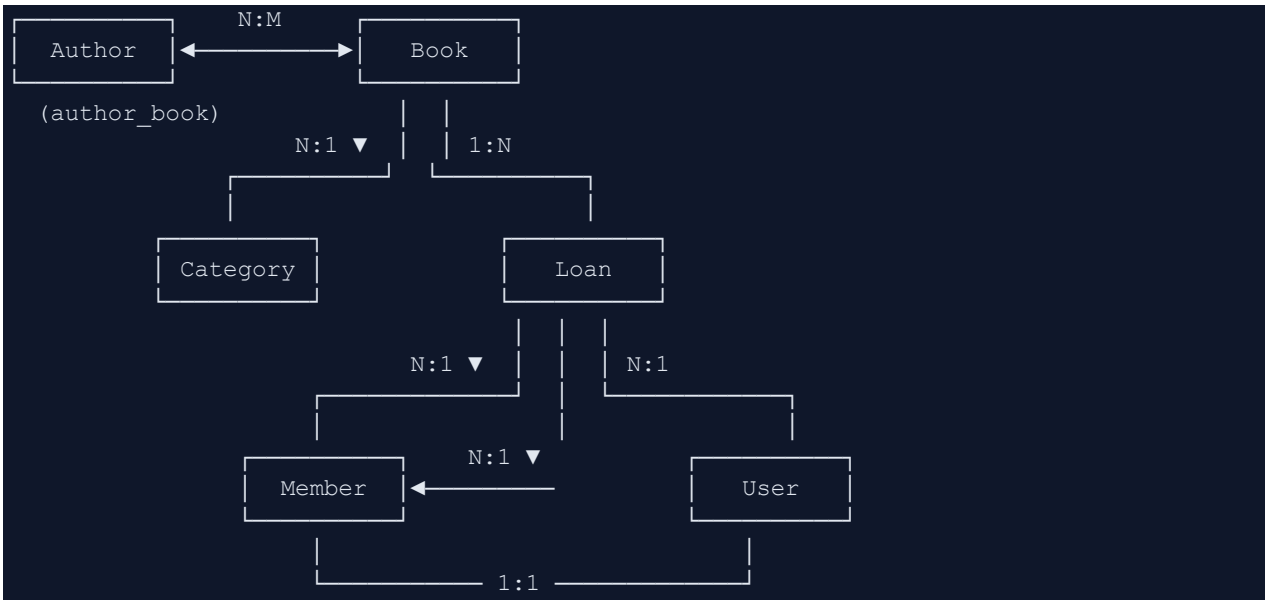
El modelo User ya existe en Laravel. Solo necesitamos agregar la relación con Member. Abre app/Models/User.php y agrega el siguiente método dentro de la clase:

```
// Agregar este use al inicio del archivo
use App\Models\Member;

// Agregar este método dentro de la clase User

/**
 * Relación: el usuario tiene un perfil de miembro.
 */
public function member(): \Illuminate\Database\Eloquent\Relations\HasOne
{
    return $this->hasOne(Member::class);
}
```

Relaciones Eloquent

Las relaciones definen cómo los modelos se conectan entre sí. Laravel soporta varios tipos de relaciones que reflejan las relaciones de la base de datos. En esta sección implementaremos todas las relaciones de nuestro sistema de biblioteca.

Diagrama de Relaciones



Paso 7: Relación Author ↔ Book (muchos a muchos)

> **belongsToMany**
> La relación muchos a muchos (belongsToMany) conecta dos modelos a través de una tabla pivote intermedia. En nuestro caso, la tabla author_book permite que un autor tenga muchos libros y un libro tenga muchos autores. El método withPivot() permite acceder a columnas adicionales de la tabla pivote.

Agrega a app/Models/Author.php:

```
/**
 * Relación: un autor tiene muchos libros.
 */
public function books(): \Illuminate\Database\Eloquent\Relations\BelongsToMany
{
    return $this->belongsToMany(Book::class)
        ->withPivot('role')
        ->withTimestamps();
}
```

Agrega a app/Models/Book.php:

```
/**
 * Relación: un libro tiene muchos autores.
 */
public function authors(): \Illuminate\Database\Eloquent\Relations\BelongsToMany
{
    return $this->belongsToMany(Author::class)
        ->withPivot('role')
        ->withTimestamps();
}
```

Paso 8: Relación Category → Books (uno a muchos)

> **hasMany / belongsTo**
> La relación uno a muchos (hasMany) indica que un registro padre tiene múltiples registros hijos. La relación inversa (belongsTo) conecta el hijo con su padre. Laravel determina automáticamente la clave foránea por convención: para Category, busca category_id en la tabla books.

Agrega a app/Models/Category.php:

```
/**
 * Relación: una categoría tiene muchos libros.
 */
public function books(): \Illuminate\Database\Eloquent\Relations\HasMany
{
    return $this->hasMany(Book::class);
}
```

Agrega a app/Models/Book.php:

```
/**
 * Relación: un libro pertenece a una categoría.
 */
public function category(): \Illuminate\Database\Eloquent\Relations\BelongsTo
{
    return $this->belongsTo(Category::class);
}
```

Paso 9: Relación User ↔ Member (uno a uno)

> **hasOne / belongsTo**
> La relación uno a uno (hasOne) indica que un registro tiene exactamente un registro relacionado. En nuestro caso, un usuario puede tener un solo perfil de miembro, y cada perfil de miembro pertenece a un solo usuario.

La relación en User ya fue definida en el Paso 6. Ahora agrega la relación inversa en app/Models/Member.php:

```
/**
 * Relación: un miembro pertenece a un usuario.
 */
public function user(): \Illuminate\Database\Eloquent\Relations\BelongsTo
{
    return $this->belongsTo(User::class);
}
```

Paso 10: Relación Member → Loans (uno a muchos)

Agrega a app/Models/Member.php:

```
/**
 * Relación: un miembro tiene muchos préstamos.
 */
public function loans(): \Illuminate\Database\Eloquent\Relations\HasMany
{
    return $this->hasMany(Loan::class);
}

/**
 * Relación: préstamos activos del miembro.
```

```
 */
public function activeLoans(): \Illuminate\Database\Eloquent\Relations\HasMany
{
    return $this->hasMany(Loan::class)->where('status', 'active');
}
```

Agrega a app/Models/Loan.php:

```
/**
 * Relación: un préstamo pertenece a un miembro.
 */
public function member(): \Illuminate\Database\Eloquent\Relations\BelongsTo
{
    return $this->belongsTo(Member::class);
}
```

Paso 11: Relación Book → Loans

Agrega a app/Models/Book.php:

```
/**
 * Relación: un libro tiene muchos préstamos.
 */
public function loans(): \Illuminate\Database\Eloquent\Relations\HasMany
{
    return $this->hasMany(Loan::class);
}

/**
 * Relación: préstamos activos del libro.
 */
public function activeLoans(): \Illuminate\Database\Eloquent\Relations\HasMany
{
    return $this->hasMany(Loan::class)->where('status', 'active');
}
```

Paso 12: Relación Loan → User (bibliotecario)

Agrega a app/Models/Loan.php:

```
/**
 * Relación: el préstamo fue realizado por un bibliotecario.
 */
public function librarian(): \Illuminate\Database\Eloquent\Relations\BelongsTo
{
    return $this->belongsTo(User::class, 'loaned_by');
}

/**
 * Relación: un préstamo pertenece a un libro.
 */
public function book(): \Illuminate\Database\Eloquent\Relations\BelongsTo
{
    return $this->belongsTo(Book::class);
}
```

> **Clave foránea personalizada**
> Cuando la clave foránea no sigue la convención de Laravel (model_id), debemos especificarla como segundo argumento. En este caso, la columna loaned_by referencia a la tabla users, por eso pasamos 'loaned_by' en belongsTo(User::class, 'loaned_by').

Resumen de Relaciones

Modelo Origen	Método	Tipo	Modelo Destino
Author	books()	belongsToMany	Book
Book	authors()	belongsToMany	Author
Category	books()	hasMany	Book
Book	category()	belongsTo	Category
User	member()	hasOne	Member
Member	user()	belongsTo	User
Member	loans()	hasMany	Loan
Member	activeLoans()	hasMany (filtrado)	Loan
Book	loans()	hasMany	Loan
Book	activeLoans()	hasMany (filtrado)	Loan
Loan	member()	belongsTo	Member
Loan	book()	belongsTo	Book
Loan	librarian()	belongsTo	User

Factories y Seeders

Las factories generan datos falsos pero realistas para pruebas. Los seeders ejecutan las factories en el orden correcto para poblar la base de datos con datos coherentes.

> **Factory**
> Una factory es una clase que define cómo generar instancias de un modelo con datos de prueba. Utiliza la librería Faker para generar nombres, direcciones, textos y otros datos realistas. Cada factory tiene un método definition() que retorna un arreglo con los campos del modelo.

Paso 13: AuthorFactory

```
php artisan make:factory AuthorFactory
```

Edita database/factories/AuthorFactory.php:

```
<?php

namespace Database\Factories;

use App\Models\Author;
use Illuminate\Database\Eloquent\Factories\Factory;

class AuthorFactory extends Factory
{
    protected $model = Author::class;

    public function definition(): array
    {
        $nationalities = [
            'Bolivia', 'Argentina', 'Colombia',
            'México', 'Perú', 'Chile',
            'Ecuador', 'Venezuela', 'España',
            'Uruguay', 'Paraguay', 'Brasil',
        ];

        return [
            'first_name'  => fake()->firstName(),
            'last_name'   => fake()->lastName(),
            'nationality' => fake()->randomElement($nationalities),
            'birth_date'  => fake()->dateTimeBetween(
                '-100 years', '-20 years'
            ),
            'biography'   => fake()->paragraphs(2, true),
        ];
    }
}
```

Paso 14: CategorySeeder (categorías predefinidas)

Para las categorías usaremos un seeder con datos predefinidos en lugar de una factory, porque queremos categorías específicas y consistentes:

```
php artisan make:seeder CategorySeeder
```

Edita database/seeders/CategorySeeder.php:

```php
<?php

namespace Database\Seeders;

use App\Models\Category;
use Illuminate\Database\Seeder;

class CategorySeeder extends Seeder
{
    public function run(): void
    {
        $categories = [
            ['name' => 'Novela',          'color' => '#3B82F6',
             'description' => 'Obras de ficción narrativa'],
            ['name' => 'Ciencia Ficción', 'color' => '#8B5CF6',
             'description' => 'Ficción especulativa y futurista'],
            ['name' => 'Historia',        'color' => '#EF4444',
             'description' => 'Obras sobre eventos históricos'],
            ['name' => 'Poesía',          'color' => '#EC4899',
             'description' => 'Obras en verso y prosa poética'],
            ['name' => 'Ciencia',         'color' => '#10B981',
             'description' => 'Divulgación y textos científicos'],
            ['name' => 'Tecnología',      'color' => '#F59E0B',
             'description' => 'Computación, ingeniería y más'],
            ['name' => 'Filosofía',       'color' => '#6366F1',
             'description' => 'Pensamiento filosófico y ética'],
            ['name' => 'Arte',            'color' => '#F97316',
             'description' => 'Artes visuales, música, teatro'],
            ['name' => 'Biografía',       'color' => '#14B8A6',
             'description' => 'Biografías y autobiografías'],
            ['name' => 'Infantil',        'color' => '#A855F7',
             'description' => 'Literatura para niños y jóvenes'],
        ];

        foreach ($categories as $data) {
            Category::create($data);
        }
    }
}
```

> **Slug automático**
> No necesitamos definir el slug porque el método boot() del modelo Category lo genera automáticamente a partir del nombre al crear cada registro.

Paso 15: BookFactory

```
php artisan make:factory BookFactory
```

Edita database/factories/BookFactory.php:

```php
<?php

namespace Database\Factories;

use App\Models\Book;
use Illuminate\Database\Eloquent\Factories\Factory;

class BookFactory extends Factory
```

```
{
    protected $model = Book::class;

    public function definition(): array
    {
        $totalCopies = fake()->numberBetween(1, 10);

        return [
            'title'            => ucfirst(fake()->sentence(3)),
            'isbn'             => fake()->isbn13(),
            'publisher'        => fake()->company(),
            'publish_year'     => fake()->numberBetween(1950, 2025),
            'pages'            => fake()->numberBetween(80, 1200),
            'language'         => fake()->randomElement([
                'Español', 'Español', 'Español', 'Español',
                'Inglés',
            ]),
            'description'      => fake()->paragraph(),
            'cover_url'        => null,
            'total_copies'     => $totalCopies,
            'available_copies' => $totalCopies,
            'status'           => 'available',
            'category_id'      => null, // Se asigna en el seeder
        ];
    }
}
```

Paso 16: MemberFactory

```
php artisan make:factory MemberFactory
```

Edita database/factories/MemberFactory.php:

```
<?php

namespace Database\Factories;

use App\Models\Member;
use Illuminate\Database\Eloquent\Factories\Factory;

class MemberFactory extends Factory
{
    protected $model = Member::class;

    public function definition(): array
    {
        $type = fake()->randomElement([
            'standard', 'premium', 'student',
        ]);

        $maxLoans = match ($type) {
            'premium' => 7,
            'student' => 2,
            default   => 3,
        };

        return [
            'user_id'               => null, // Se asigna en el seeder
            'phone'                 => fake()->phoneNumber(),
```

```
            'address'               => fake()->address(),
            'membership_type'       => $type,
            'membership_expires_at' => fake()->dateTimeBetween(
                'now', '+1 year'
            ),
            'max_loans'             => $maxLoans,
            'is_active'             => true,
        ];
    }
}
```

Paso 17: LoanFactory

```
php artisan make:factory LoanFactory
```

Edita database/factories/LoanFactory.php:

```
<?php

namespace Database\Factories;

use App\Models\Loan;
use Illuminate\Database\Eloquent\Factories\Factory;

class LoanFactory extends Factory
{
    protected $model = Loan::class;

    public function definition(): array
    {
        $loanDate    = fake()->dateTimeBetween('-60 days', 'now');
        $dueDate     = (clone $loanDate)->modify('+14 days');
        $isReturned  = fake()->boolean(30); // 30% devuelto

        $returnedDate = $isReturned
            ? fake()->dateTimeBetween($loanDate, 'now')
            : null;

        $status = $isReturned
            ? 'returned'
            : ($dueDate < now() ? 'overdue' : 'active');

        return [
            'book_id'       => null, // Se asigna en el seeder
            'member_id'     => null, // Se asigna en el seeder
            'loaned_by'     => null, // Se asigna en el seeder
            'loan_date'     => $loanDate,
            'due_date'      => $dueDate,
            'returned_date' => $returnedDate,
            'status'        => $status,
            'notes'         => fake()->optional(0.3)->sentence(),
        ];
    }
}
```

Paso 18: DatabaseSeeder

Ahora configuramos el seeder principal que ejecuta todo en el orden correcto. Edita database/seeders/DatabaseSeeder.php:

```php
<?php

namespace Database\Seeders;

use App\Models\Author;
use App\Models\Book;
use App\Models\Loan;
use App\Models\Member;
use App\Models\User;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        // 1. Crear 10 usuarios
        $users = User::factory(10)->create();

        // 2. Crear 15 autores
        $authors = Author::factory(15)->create();

        // 3. Ejecutar CategorySeeder (10 categorías)
        $this->call(CategorySeeder::class);
        $categoryIds = \App\Models\Category::pluck('id');

        // 4. Crear 50 libros con categoría random
        $books = Book::factory(50)->create([
            'category_id' => fn () => $categoryIds->random(),
        ]);

        // 5. Asignar 1-3 autores a cada libro
        $books->each(function (Book $book) use ($authors) {
            $randomAuthors = $authors->random(
                rand(1, 3)
            );
            $book->authors()->attach(
                $randomAuthors->pluck('id'),
                ['role' => 'Autor']
            );
        });

        // 6. Crear miembros para los primeros 8 usuarios
        $members = $users->take(8)->map(
            fn (User $user) => Member::factory()->create([
                'user_id' => $user->id,
            ])
        );

        // 7. Crear 20 préstamos
        $memberIds = $members->pluck('id');
        $bookIds   = $books->pluck('id');

        Loan::factory(20)->create([
            'book_id'   => fn () => $bookIds->random(),
            'member_id' => fn () => $memberIds->random(),
            'loaned_by' => fn () => $users->random()->id,
        ]);
    }
}
```

Ejecutar los Seeders

Para poblar la base de datos, ejecuta los siguientes comandos:

```
# Resetear migraciones y ejecutar seeders
php artisan migrate:fresh --seed
```

> **Advertencia**
> El comando migrate:fresh elimina TODAS las tablas y las vuelve a crear. Nunca lo ejecutes en un entorno de producción. Es ideal para desarrollo, ya que garantiza un estado limpio de la base de datos.

Verifica los datos en PostgreSQL:

```
# Conectar a PostgreSQL
psql -U tu_usuario -d library_app

-- Verificar conteos
SELECT COUNT(*) AS total_users FROM users;
SELECT COUNT(*) AS total_authors FROM authors;
SELECT COUNT(*) AS total_categories FROM categories;
SELECT COUNT(*) AS total_books FROM books;
SELECT COUNT(*) AS total_members FROM members;
SELECT COUNT(*) AS total_loans FROM loans;
SELECT COUNT(*) AS total_pivotes FROM author_book;
```

Tinker: Explorar Relaciones

Tinker es la consola interactiva de Laravel (basada en PsySH). Permite ejecutar código PHP directamente, lo que resulta ideal para probar modelos y relaciones sin necesidad de crear rutas o controladores.

Paso 19: Abrir Tinker y explorar

```
php artisan tinker
```

Ejecuta las siguientes consultas una por una:

Buscar un autor y listar sus libros:

```
use App\Models\Author;
$author = Author::first();
$author->full_name;
$author->books;
```

Buscar un libro y sus autores:

```
use App\Models\Book;
$book = Book::first();
$book->title;
$book->authors;
```

Ver el rol en la tabla pivote:

```
$book->authors->first()->pivot->role;
```

Buscar libros de una categoría:

```
use App\Models\Category;
$cat = Category::where('name', 'Novela')->first();
$cat->books->count();
$cat->books->pluck('title');
```

Préstamos activos de un miembro:

```
use App\Models\Member;
$member = Member::first();
$member->loans->count();
$member->activeLoans;
```

Usuario → Miembro:

```
use App\Models\User;
$user = User::has('member')->first();
$user->member;
$user->member->member_code;
```

Conteo de libros disponibles:

```
Book::where('status', 'available')->count();
```

Conteo de préstamos activos:

```
use App\Models\Loan;
Loan::where('status', 'active')->count();
```

> **Ejercicio en Tinker**
> Prueba también las siguientes consultas:
> - $book->isAvailable (accessor de disponibilidad)
> - $loan = Loan::where('status', 'active')->first(); $loan->isOverdue
> - $loan->daysRemaining

```
- $member->isMembershipActive
```

Git: Confirmar y Etiquetar

Paso 20: Commit y tag v2.0

Confirma todos los cambios y crea el tag de la guía:

```
git add .
git commit -m "feat: modelos con relaciones, factories y seeders"

git tag -a v2.0 -m "Guía 5: Modelos y relaciones"
git push origin main --tags
```

Ejercicios Propuestos

Ejercicio 1: Modelo Fine (Multa)

Crea un modelo Fine que represente multas por devoluciones tardías. Implementa lo siguiente:

Aspecto	Detalle
Tabla	fines
Campos	id, loan_id, amount (decimal), reason, status (pending/paid), paid_at (nullable)
Relaciones	Loan hasMany Fines, Fine belongsTo Loan
Accessor	getIsPendingAttribute(): retorna true si status === 'pending'
Factory	FineFactory con amount entre 5.00 y 50.00, reason con fake()->sentence()
Seeder	Crear multas para todos los préstamos con status 'overdue'

Ejercicio 2: Modelo BookReview (Reseña)

Crea un modelo BookReview para que los miembros puedan reseñar libros:

Aspecto	Detalle
Tabla	book_reviews
Campos	id, book_id, member_id, rating (1-5), title, body, timestamps
Relaciones	Book hasMany BookReviews, Member hasMany BookReviews
Accessor en Book	getAverageRatingAttribute(): $this->reviews()->avg('rating')
Factory	BookReviewFactory con rating entre 1-5, title con sentence(), body con paragraph()
Seeder	Crear 30 reviews random asociadas a libros y miembros existentes

Ejercicio 3: Verificación en Tinker

Abre Tinker y verifica que las nuevas relaciones funcionan correctamente:

```
// Verificar reseñas de un libro
$book = Book::has('reviews')->first();
$book->averageRating;
$book->reviews->count();

// Verificar reseñas de un miembro
$member = Member::has('bookReviews')->first();
$member->bookReviews;

// Verificar multas de un préstamo
$loan = Loan::has('fines')->first();
$loan->fines;
$loan->fines->first()->isPending;
```

Rúbrica de Evaluación

La evaluación de esta guía se basa en los siguientes criterios (sobre 100 puntos):

Criterio	Puntos	Descripción
Modelos Eloquent	20	Cada modelo tiene $fillable, $casts y accessors correctos. Boot methods generan códigos/slugs automáticamente.
Relaciones Eloquent	25	belongsToMany con withPivot, hasMany, belongsTo y hasOne implementadas correctamente. Relaciones filtradas (activeLoans) funcionales.
Factories	15	Datos realistas con Faker. Nacionalidades latinoamericanas, tipos de membresía con max_loans coherentes, préstamos con fechas lógicas.
Seeders	15	DatabaseSeeder ejecuta en orden correcto. Datos coherentes: libros con categorías, autores en tabla pivote, miembros con usuarios, préstamos válidos.
Verificación Tinker	10	Capturas de pantalla o evidencia de las consultas ejecutadas en Tinker mostrando las relaciones funcionando.
Ejercicios propuestos	10	Modelos Fine y BookReview implementados con factories, seeders y verificación en Tinker.
Git tag v2.0	5	Commit con mensaje descriptivo y tag v2.0 creado y publicado correctamente.

Entregables

Para la entrega de esta guía, asegúrate de incluir:

- ✓ Repositorio Git con tag v2.0 visible y accesible.
- ✓ Todos los modelos creados en app/Models/ con $fillable, $casts, accessors y relaciones.
- ✓ Factories en database/factories/ con datos realistas.
- ✓ Seeders funcionales en database/seeders/ que ejecuten sin errores.
- ✓ Ejercicios propuestos (Fine y BookReview) implementados y verificados.
- ✓ Capturas de pantalla de las consultas en Tinker mostrando las relaciones.